PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
Software Engineering (UE24CS341A) — Level 3 Mini-Project | Semester 5, Section G
DELIVERABLE 1 – SYNOPSIS / PROJECT PROPOSAL

1.1 Project Title and Group

Field	Entry
Project Title	StudentRMS – Student Record Management System
Group Number	Team 4
Project ID	SRMS-2026-04
Technology	(console application, file-based persistence)
Complexity	Easy
Customer / Stakeholder	Department Academic Office (record-keeping staff)

1.2 Team Profile and Product Ownership

To strictly align with the requirement that each member must own at least one complete functional feature, the four functional areas of StudentRMS are split equally below. Each member owns one feature end-to-end and is individually responsible for the matching sections of both the Synopsis and the SRS.

Member	SRN	Owned Functional Feature	Synopsis Deliverables	SRS Deliverables
Member 1 — Samiksha Jayanth	**PES1UG24CS414**	**Administrator Authentication & Session Management**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Authentication ● Executive Summary / Abstract	**SRS Deliverables:** ● Section 2.2: Purpose (Introduction) ● Section 2.4: User Classes & Characteristics ● Section 2.7: Security Requirements (SEC-001, SEC-002, SEC-005) ● FRs: FR-001 (Login/ Authentication) ● Use Cases: UC-01 (Administrator Login)
Member 2 — Rohan M	**PES1UG24CS381**	**Record Search, View & Listing**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Search/Retrieval ● High-Level Scope & Project Boundaries	**SRS Deliverables:** ● Section 2.3: Scope (In Scope / Out of Scope) ● Section 2.6: NFR-001 (Performance) ● FRs: FR-004 (Search), FR-007 (List & Sort) ● Use Cases: UC-03 (Search/ View Record), UC-06 (List & Sort Records)

Member	SRN	Owned Functional Feature	Synopsis Deliverables	SRS Deliverables
Member 3 — Saksham Bajpayi		**Record Lifecycle Management (Add, Update, Delete)**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Record Management ● Key Deliverables & Expected Outcomes	**SRS Deliverables:** ● Section 2.5: Functional Requirements (detailed FRs) ● Section 2.6: NFR-002 (Reliability) ● FRs: FR-002 (Add), FR-003 (Duplicate Check), FR-005 (Update), FR-006 (Delete), FR-012 (Validation) ● Use Cases: UC-02 (Add Record), UC-04 (Update Record), UC-05 (Delete Record)
Member 4 — Riddhi Mundada	**PES1UG24CS372**	**Reporting, CGPA Computation & Data Persistence**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Reporting/ Persistence ● Target Audience & System Constraints	**SRS Deliverables:** ● Section 2.2: Design & Implementation Constraints (file-based storage) ● Section 2.12: Tool Stack ● FRs: FR-008 (CGPA Calc), FR-009 (Rank List), FR-010 (Persistence), FR-011 (Export) ● Use Cases: UC-07 (Generate Rank List / Report)

Important: UI design, file-handling support and unit testing are supporting responsibilities, not substitutes for owning a functional feature, per the project guidelines.

1.3 Problem Statement

Academic departments that still track student information — personal details, subject-wise marks, and computed CGPA — using registers or unstructured spreadsheets face frequent errors, duplicate entries, slow retrieval, and no easy way to generate rank lists or class reports. StudentRMS digitizes this workflow as a lightweight C11 console application backed by persistent file storage, giving an authenticated administrator fast, validated, and auditable control over student records.

1.4 Objectives

- Provide a single authenticated entry point so only authorized staff can modify records.
- Eliminate duplicate/invalid student entries through validation at data-entry time.
- Allow fast retrieval of any student's record by roll number or name.
- Automatically compute percentage/CGPA instead of manual calculation.
- Generate a sorted class list and rank list on demand.
- Persist all data to disk so records survive across program runs and system restarts.
- Develop the product incrementally using Agile practices.

1.5 Functional Features

- Administrator login/authentication before any record operation is permitted.
- Add a new student record (roll number, name, department, semester, subject marks).
- Search / view a student record by roll number or name.
- Update an existing student record.
- Delete an existing student record (with confirmation).
- List all student records, sortable by roll number, name, or CGPA.

- Automatic percentage/CGPA calculation from stored subject marks.
- Generate and export a class rank list / report to a text file.
- Persistent file-based storage so data is retained between sessions.

1.6 Short-Term Plan

Iteration	Planned Output	Owner	Acceptance Evidence
Sprint 1	Project setup, requirements baseline, data structures, admin login	All / Samiksha	User stories + Git repo + working login
Sprint 2	Search/view/listing module (by roll no. and name, sortable)	Rohan	Demo + unit tests
Sprint 3	Add-record module with input validation	Saksham	Demo + unit tests
Sprint 4	Update & delete modules with confirmation	Saksham	Integration tests + PRs
Sprint 5	CGPA calculation, rank-list, report generation & file persistence	Riddhi	System demo + test evidence
Sprint 6	System-wide validation, integration testing and release	All	Final report + deployment/demo

1.7 Synopsis Submission Checklist

☐ Title, Group Number and Project ID

☐ All team members: names and SRNs

☐ Problem statement and customer/stakeholder

☐ Intended users

☐ Objectives and scope

☐ Functional features

☐ Team ownership matrix

☐ Short-term plan

☐ Proposed technology/tool stack

☐ Agile justification

DELIVERABLE 1 — SYNOPSIS CORRECTIONS

StudentRMS — Student Record Management System | Team 4 | PES University

This addendum is attached to the original three-page synopsis. The corrections below supersede the corresponding incorrect entries in the source PDF.

1. Corrections applied

Location	Original entry	Corrected entry
Page 1 — Technology	C / C++	C11 (console application, file-based persistence)
Page 2 — Member 3, Saksham Bajpayi	PES1UG24CS414	PES1UG24CS404
Page 2 — Problem Statement	C/C++ console application	C11 console application

2. Agile development justification

An Agile approach is appropriate because the application can be divided into independently testable feature areas: authentication, search/listing, record lifecycle, and reporting/persistence. Short iterations allow each feature to be implemented and integrated incrementally. Frequent builds, automated regression tests, and sprint reviews expose integration defects early and provide visible progress against the project backlog. The repository records two executed sprints; the six-sprint schedule in the initial proposal should be described as the original estimate, not as six completed sprints.

3. Synopsis submission checklist

Checklist item	Disposition
Project title, group, project ID and stakeholder identified	Present in source synopsis
Problem statement and project objectives	Present in source synopsis
Technology matches implemented product	Corrected to C11
Team ownership and feature responsibilities	Member 3 SRN corrected; feature ownership retained
Scope and system boundaries	Present in source synopsis
Expected outcomes and deliverables	Review against the final repository before marking complete
Agile approach and sprint plan	Justification added; distinguish planned from executed sprints

Submission note: Do not mark any checklist item complete unless it is supported by the submitted synopsis and repository evidence. The original PDF's blank checklist is not silently treated as completed.